MACHINE LEARNING E-FILE

Name : Shahid Firozabad

RollNo : 2305029

Class : TYBCA

Batch : 02

Subject : Machine Learning

College : Spes’s Goa Multi Faculty College

EXPERIMENT 01

Questions

1) Merging Data Sources – Excel File

Code

```
import pandas as pd
df1 = pd.read_excel("../Dataset/students_data.xlsx")
df2 = pd.read_excel("../Dataset/students_data.xlsx")
merged = pd.merge(df1, df2, on="Student_ID")
print(merged.head())
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q1.py
   Student_ID  Age_x  Study_Hours_x  Attendance_x  ...  Previous_Score_y  Final_Score_y Gender_y Passed_y
0           1     19              2            80  ...                60             65        M      Yes
1           2     20              3            85  ...                65             70        F      Yes
2           3     21              1            60  ...                50             55        M       No
3           4     19              4            90  ...                70             78        F      Yes
4           5     22              5            95  ...                75             85        M      Yes

[5 rows x 15 columns]
```

2) Handling Missing Value

Code

```
import pandas as pd
df = pd.read_csv("../Dataset/students_data.csv")
df.loc[3, "Study_Hours"] = None
missing = pd.DataFrame(df.isnull().sum(), columns=["Missing Values"])
print("Missing Value Table")
print(missing)
df = df.fillna(df.mean(numeric_only=True))
print("\nCleaned Dataset")
print(df)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q2.py
Missing Value Table
                 Missing Values
Student_ID                    0
Age                           0
Study_Hours                   1
Attendance                    0
Previous_Score                0
Final_Score                   0
Gender                        0
Passed                        0

Cleaned Dataset
    Student_ID  Age  Study_Hours  Attendance  Previous_Score  Final_Score Gender Passed
0            1   19     2.000000          80              60           65      M    Yes
1            2   20     3.000000          85              65           70      F    Yes
2            3   21     1.000000          60              50           55      M     No
3            4   19     3.214286          90              70           78      F    Yes
4            5   22     5.000000          95              75           85      M    Yes
5            6   20     2.000000          70              55           60      F     No
6            7   23     6.000000          98              80           90      M    Yes
7            8   21     3.000000          88              67           72      F    Yes
8            9   20     4.000000          92              72           80      M    Yes
9           10   22     2.000000          65              58           62      F     No
10          11   19     1.000000          55              45           50      M     No
11          12   24     7.000000          99              85           95      F    Yes
12          13   21     3.000000          82              66           71      M    Yes
13          14   23     4.000000          90              73           79      F    Yes
14          15   20     2.000000          75              60           64      M    Yes

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
```

3) Merging Data Sources – CSV File (Merge, Concat, Join)

Code

```
import pandas as pd
df1 = pd.read_csv("../Dataset/students_data.csv")
df2 = pd.read_csv("../Dataset/students_data.csv")
merge = pd.merge(df1, df2, on="Student_ID")
concat = pd.concat([df1, df2])
join = df1.join(df2, lsuffix="_left", rsuffix="_right")
print("MERGE RESULT")
print(merge.head())
print("\nCONCAT RESULT")
print(concat.head())
print("\nJOIN RESULT")
print(join.head())
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q3.py
MERGE RESULT
   Student_ID  Age_x  Study_Hours_x  Attendance_x  Previous_Score_x  ...  Attendance_y Previous_Score_y Final_Score_y  Gender_y  Passed_y
0           1     19              2            80                60  ...            80               60            65         M       Yes
1           2     20              3            85                65  ...            85               65            70         F       Yes
2           3     21              1            60                50  ...            60               50            55         M        No
3           4     19              4            90                70  ...            90               70            78         F       Yes
4           5     22              5            95                75  ...            95               75            85         M       Yes

[5 rows x 15 columns]

CONCAT RESULT
   Student_ID  Age  Study_Hours  Attendance  Previous_Score  Final_Score Gender Passed
0           1   19            2          80              60           65      M    Yes
1           2   20            3          85              65           70      F    Yes
2           3   21            1          60              50           55      M     No
3           4   19            4          90              70           78      F    Yes
4           5   22            5          95              75           85      M    Yes

JOIN RESULT
   Student_ID_left  Age_left  Study_Hours_left  Attendance_left  ...  Previous_Score_right  Final_Score_right Gender_right Passed_right
0                1        19                 2               80  ...                    60                 65           M          Yes
1                2        20                 3               85  ...                    65                 70           F          Yes
2                3        21                 1               60  ...                    50                 55           M           No
3                4        19                 4               90  ...                    70                 78           F          Yes
4                5        22                 5               95  ...                    75                 85           M          Yes

[5 rows x 16 columns]

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
```

4) Outlier Detection Using IQR

Code

```
import pandas as pd
df = pd.read_csv("../Dataset/students_data.csv")
Q1 = df["Study_Hours"].quantile(0.25)
Q3 = df["Study_Hours"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df["Study_Hours"] < lower) | (df["Study_Hours"] > upper)]
print(outliers)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q4.py
Outliers found:
   Student_ID  Age  Study_Hours  Attendance  Previous_Score  Final_Score Gender Passed
2          12   24           20          99              85           95      F    Yes

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
```

5) Data wrangling and Feature Scaling

Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
df = pd.read_csv("../Dataset/students_data.csv")
scaler = MinMaxScaler()
df[["Age","Study_Hours","Attendance","Previous_Score","Final_Score"]] =
scaler.fit_transform(
df[["Age","Study_Hours","Attendance","Previous_Score","Final_Score"]])
print(df.head())
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q5.py
   Student_ID  Age  Study_Hours  Attendance  Previous_Score  Final_Score Gender Passed
0           1  0.0     0.166667    0.568182           0.375     0.333333      M    Yes
1           2  0.2     0.333333    0.681818           0.500     0.444444      F    Yes
2           3  0.4     0.000000    0.113636           0.125     0.111111      M     No
3           4  0.0     0.500000    0.795455           0.625     0.622222      F    Yes
4           5  0.6     0.666667    0.909091           0.750     0.777778      M    Yes
```

6) Feature Correlation

Code

```
import pandas as pd
df = pd.read_csv("../Dataset/students_data.csv")
corr = df.corr(numeric_only=True)
print(corr)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q6.py
               Student_ID       Age  Study_Hours  Attendance  Previous_Score  Final_Score
Student_ID       1.000000  0.374113     0.127688    0.005768        0.139573     0.104561
Age              0.374113  1.000000     0.678247    0.465737        0.647877     0.640450
Study_Hours      0.127688  0.678247     1.000000    0.900321        0.966834     0.982484
Attendance       0.005768  0.465737     0.900321    1.000000        0.963854     0.955502
Previous_Score   0.139573  0.647877     0.966834    0.963854        1.000000     0.994513
Final_Score      0.104561  0.640450     0.982484    0.955502        0.994513     1.000000

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
```

7) Simple Linear Regression

Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
df = pd.read_csv("../Dataset/students_data.csv")
X = df[["Study_Hours"]]
y = df["Final_Score"]
model = LinearRegression()
model.fit(X,y)
pred = model.predict(X)
print(pred)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q7.py
[62.61490683 69.8136646  55.41614907 77.01242236 84.21118012 62.61490683
 91.40993789 69.8136646  77.01242236 62.61490683 55.41614907 98.60869565
 69.8136646  77.01242236 62.61490683]
```

8) Multiple Linear Regression

Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("../Dataset/students_data.csv")

X = df[["Study_Hours","Attendance","Previous_Score"]]
y = df["Final_Score"]

model = LinearRegression()
model.fit(X,y)
pred = model.predict(X)

print(pred)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q8.py
[65.26997071 71.12990105 95.45616873 76.9898314  82.84976174 59.47732759
 88.70929167 73.44675809 79.30648823 62.95071116 47.82475413 94.56842118
 72.28742863 80.46421602 65.26896966]
```

9) Naive Bayes Classifier

Code

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("../Dataset/students_data.csv")
le = LabelEncoder()

df["Gender"] = le.fit_transform(df["Gender"])
df["Passed"] = le.fit_transform(df["Passed"])

X = df[["Age","Study_Hours","Attendance","Previous_Score"]]
y = df["Passed"]

model = GaussianNB()
model.fit(X,y)
pred = model.predict(X)
print(pred)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q9.py
[1 1 1 1 1 0 1 1 1 0 0 1 1 1 0]
```

10) KNN Classifier

Code

```
# KNN Classifier
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("../Dataset/students_data.csv")

le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
df["Passed"] = le.fit_transform(df["Passed"])

X = df[["Age","Study_Hours","Attendance","Previous_Score"]]
y = df["Passed"]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X,y)
pred = model.predict(X)
print(pred)
```

Output

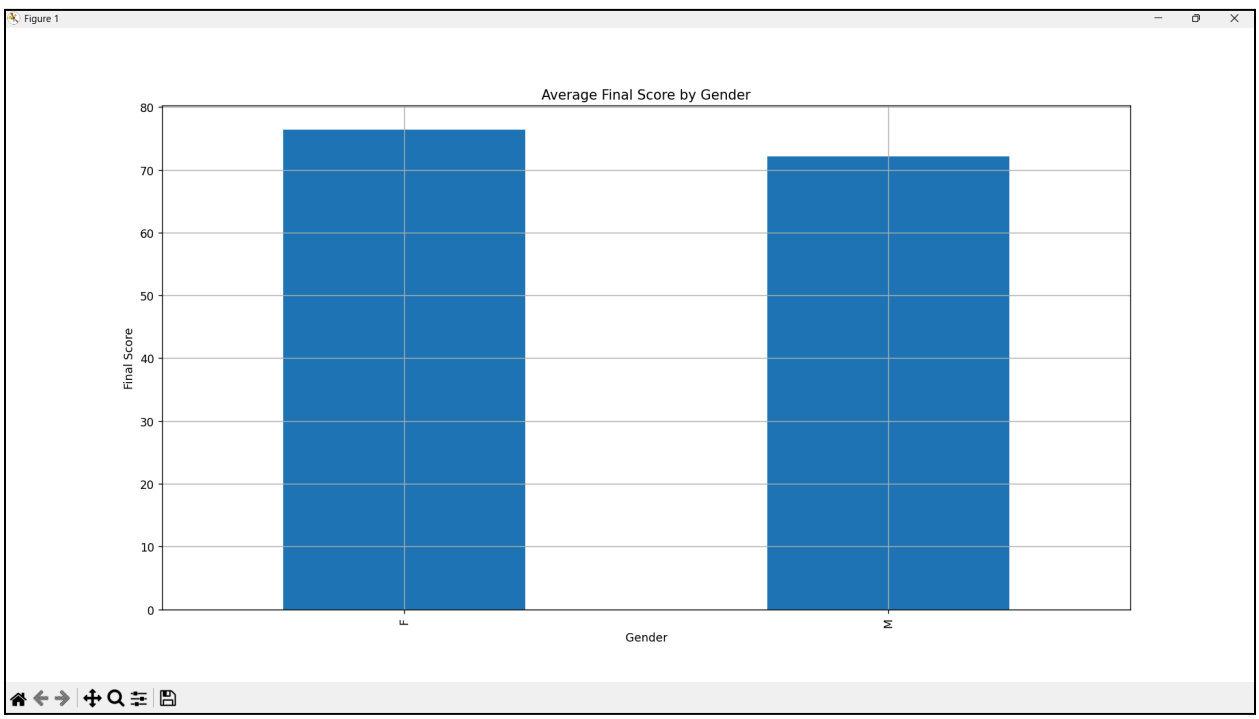
```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$ python Q10.py
[1 1 1 1 1 0 1 1 1 0 0 1 1 1 1]

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/Experiment-01
$
```

11) Bar Graph Using Python Code.

Code

```
# Bar Chart
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../Dataset/students_data.csv")
df.groupby("Gender")["Final_Score"].mean().plot(kind="bar")

plt.title("Average Final Score by Gender")
plt.xlabel("Gender")
plt.ylabel("Final Score")
plt.grid()
plt.show()
```

Output

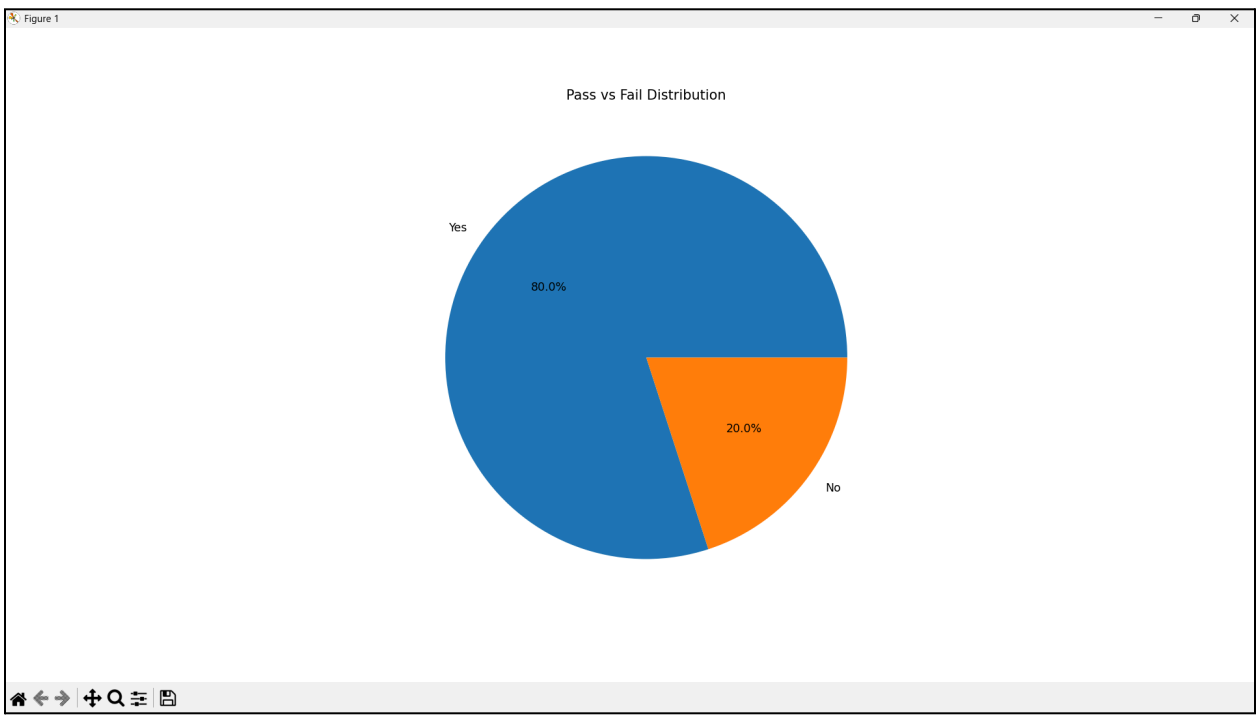


12) Pie Graph Using Python Code.

Code

```
# Pie Chart
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../Dataset/students_data.csv")

df["Passed"].value_counts().plot.pie(autopct='%1.1f%%')

plt.title("Pass vs Fail Distribution")

plt.show()
```

Output

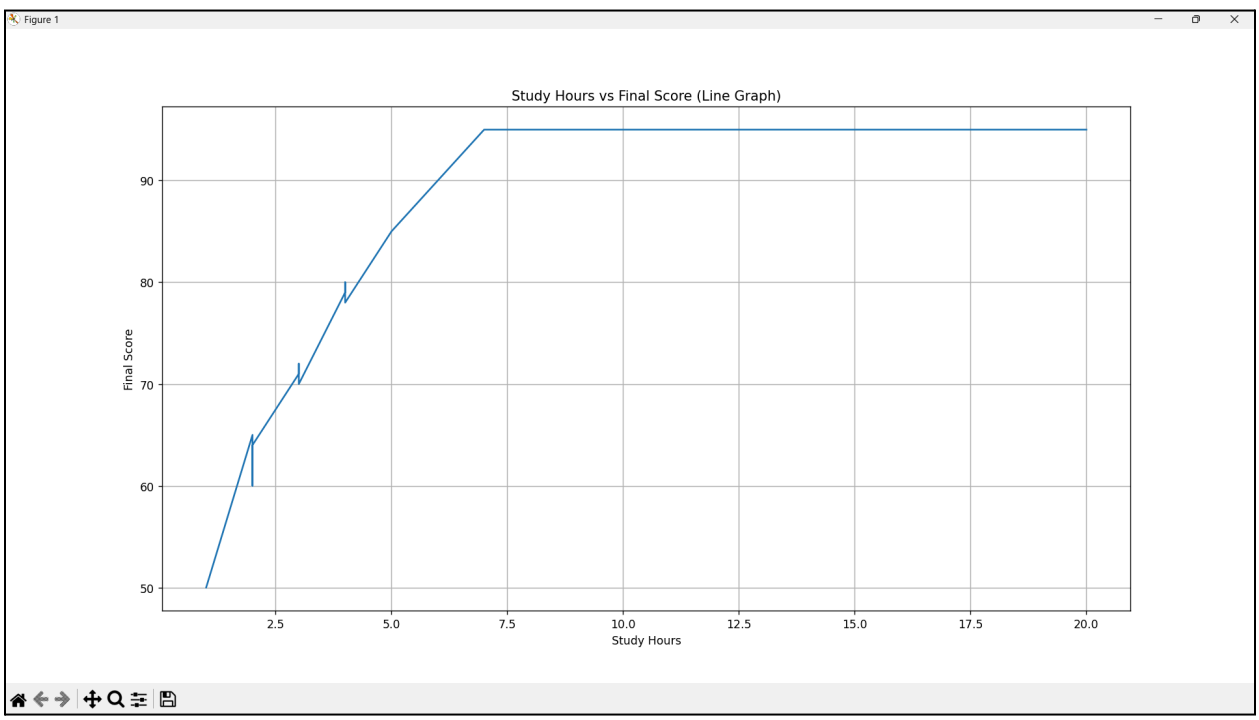


13) Line Graph Using Python Code.

Code

```
# Line Chart
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../Dataset/students_data.csv")

df = df.sort_values("Study_Hours")

plt.plot(df["Study_Hours"], df["Final_Score"])

plt.title("Study Hours vs Final Score (Line Graph)")
plt.xlabel("Study Hours")
plt.ylabel("Final Score")
plt.grid()

plt.show()
```

Output

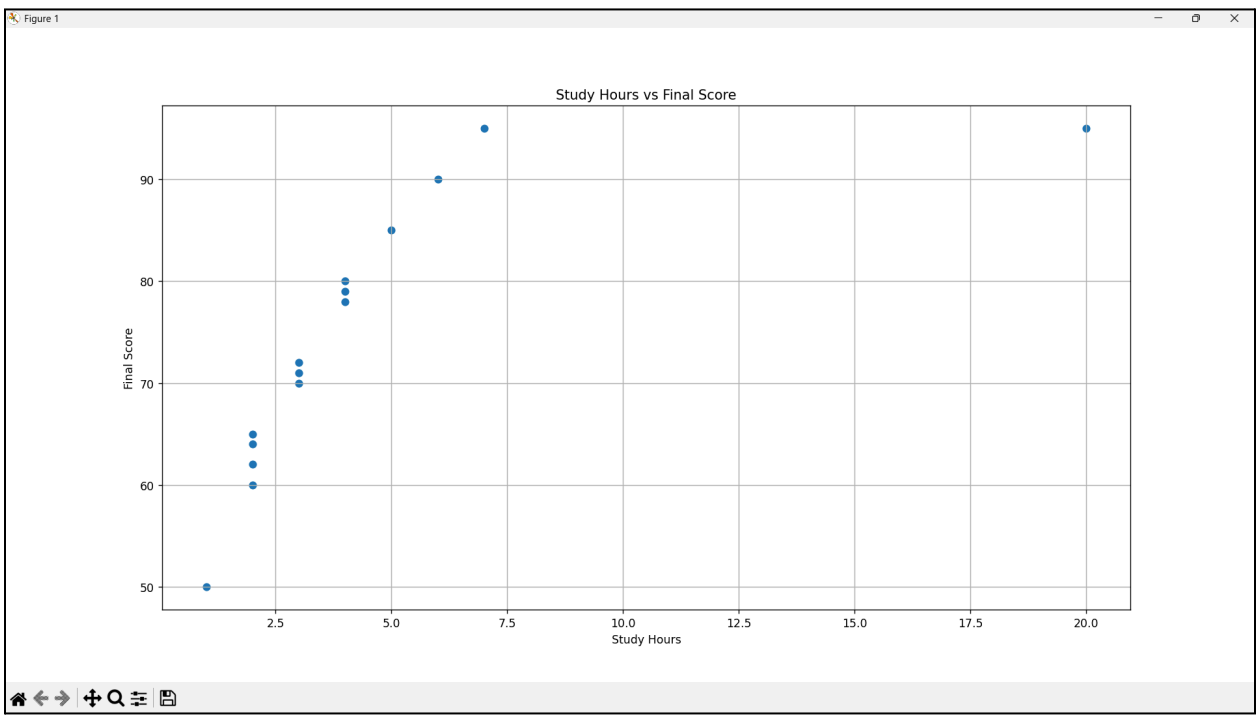


14) Scatter Graph Using Python Code.

Code

```
# Scatter Plot
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../Dataset/students_data.csv")

plt.scatter(df["Study_Hours"], df["Final_Score"])

plt.title("Study Hours vs Final Score")
plt.xlabel("Study Hours")
plt.ylabel("Final Score")
plt.grid()

plt.show()
```

Output

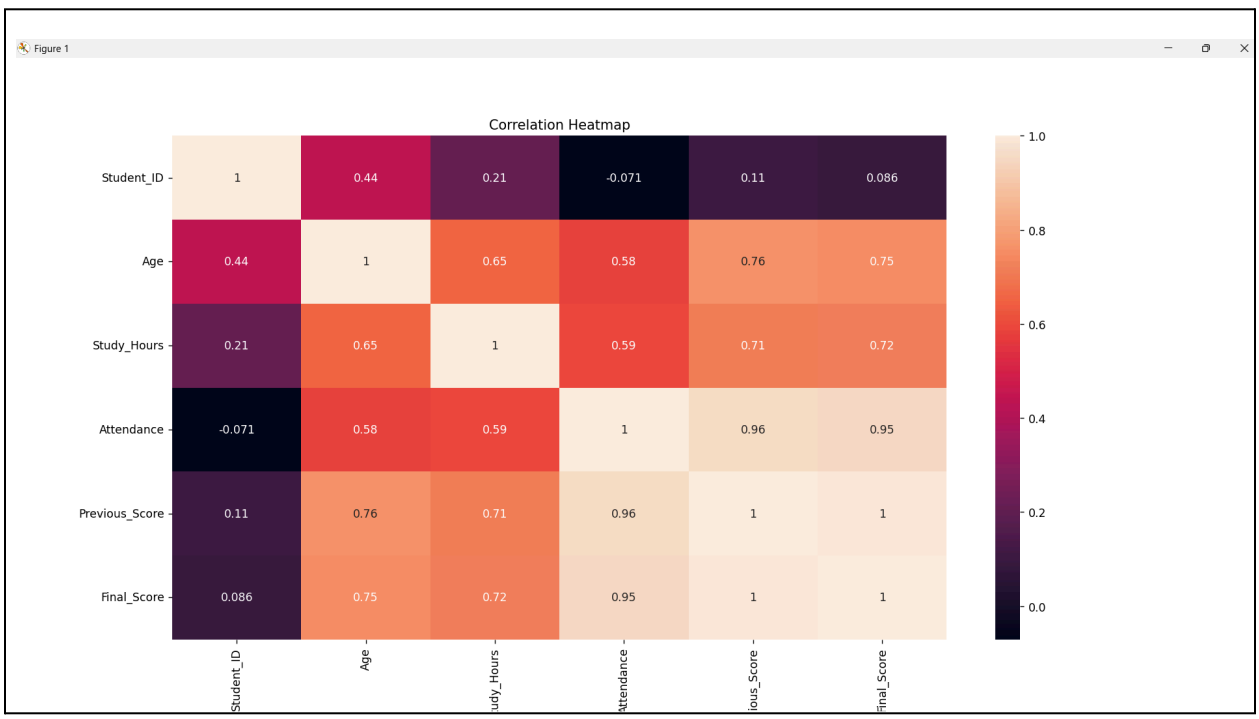


15) Heatmap Using Python Code.

Code

```
# Heatmap
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("../Dataset/students_data.csv")

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True)
plt.title("Correlation Heatmap")
plt.show()
```

Output



16) Feature Selection and Data Reduction

Code

```
# Feature Selection and Data Reduction
import pandas as pd
# Load dataset
df = pd.read_csv("../Dataset/students_data.csv")
print("Original Data:\n")
print(df.head())
# Drop unnecessary column (ID doesn't affect prediction)
df_reduced = df.drop(columns=["Student_ID"])
print("\nAfter Removing Unnecessary Column:\n")
print(df_reduced.head())
# Selecting only important features
selected_features = df[["Study_Hours", "Attendance", "Previous_Score",
"Final_Score"]]
print("\nSelected Important Features:\n")
print(selected_features.head())
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q11.py
Original Data:

   Student_ID  Age  Study_Hours  Attendance  Previous_Score  Final_Score Gender Passed
0           1   19            2          80              60           65      M    Yes
1           2   20            3          85              65           70      F    Yes
2          12   24           20          99              85           95      F    Yes
3           4   19            4          90              70           78      F    Yes
4           5   22            5          95              75           85      M    Yes

After Removing Unnecessary Column:

   Age  Study_Hours  Attendance  Previous_Score  Final_Score Gender Passed
0   19            2          80              60           65      M    Yes
1   20            3          85              65           70      F    Yes
2   24           20          99              85           95      F    Yes
3   19            4          90              70           78      F    Yes
4   22            5          95              75           85      M    Yes

Selected Important Features:

   Study_Hours  Attendance  Previous_Score  Final_Score
0            2          80              60           65
1            3          85              65           70
2           20          99              85           95
3            4          90              70           78
4            5          95              75           85

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$
```

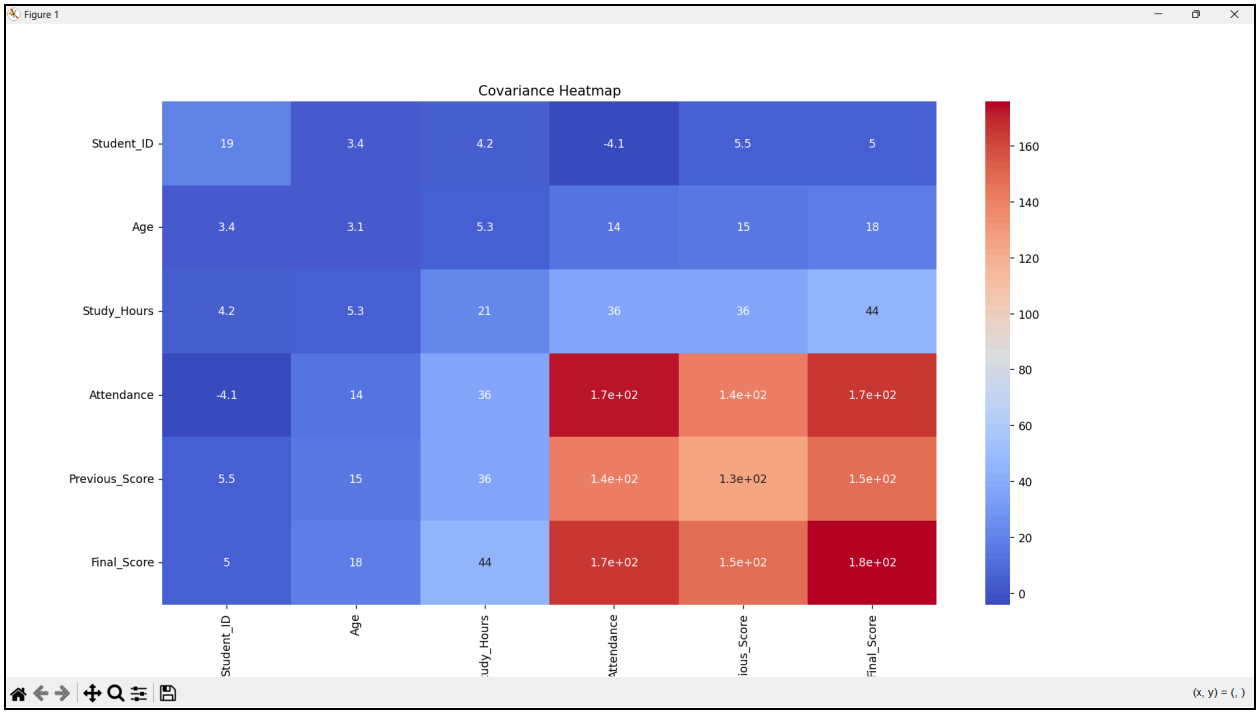
17) Covariance Feature Selection

Code

```
# Covariance Based Feature Analysis

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv("../Dataset/students_data.csv")
# Select only numeric columns
numeric_df = df.select_dtypes(include=np.number)
# Calculate covariance matrix
cov_matrix = numeric_df.cov()
print("Covariance Matrix:\n")
print(cov_matrix)
# Visualization
plt.figure(figsize=(8,6))
sns.heatmap(cov_matrix, annot=True, cmap="coolwarm")
plt.title("Covariance Heatmap")
plt.show()
```

Output

```
$ cd experiments

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q12.py
Covariance Matrix:

                Student_ID        Age  Study_Hours  Attendance  Previous_Score  Final_Score
Student_ID       18.971429   3.414286     4.157143   -4.057143        5.457143     4.957143
Age               3.414286   3.123810     5.280952   13.542857       15.109524    17.585714
Study_Hours       4.157143   5.280952    20.980952   35.600000       36.366667    43.557143
Attendance       -4.057143  13.542857    35.600000  172.742857      140.700000   165.414286
Previous_Score    5.457143  15.109524    36.366667  140.700000      125.352381   147.685714
Student_ID       18.971429   3.414286     4.157143   -4.057143        5.457143     4.957143
Age               3.414286   3.123810     5.280952   13.542857       15.109524    17.585714
Study_Hours       4.157143   5.280952    20.980952   35.600000       36.366667    43.557143
Attendance       -4.057143  13.542857    35.600000  172.742857      140.700000   165.414286
Previous_Score    5.457143  15.109524    36.366667  140.700000      125.352381   147.685714
Previous_Score    5.457143  15.109524    36.366667  140.700000      125.352381   147.685714
Final_Score       4.957143  17.585714    43.557143  165.414286      147.685714   175.685714
```



18) Feature Selection Using ANOVA Method.

Code

```
# Feature Selection using ANOVA F-Score

import pandas as pd
from sklearn.feature_selection import SelectKBest, f_classif

# Load dataset
df = pd.read_csv("../Dataset/students_data.csv")

# Convert Passed column to numeric (important step)
df["Passed"] = df["Passed"].map({"Yes": 1, "No": 0})

# Features (X) and Target (y)
X = df[["Age", "Study_Hours", "Attendance", "Previous_Score",
"Final_Score"]]
y = df["Passed"]

# Apply ANOVA F-Test
selector = SelectKBest(score_func=f_classif, k=3)
```

```python
X_new = selector.fit_transform(X, y)

# Get feature scores
scores = selector.scores_

# Display scores
feature_scores = pd.DataFrame({
    "Feature": X.columns,
    "Score": scores
})

print("Feature Scores:\n")
print(feature_scores.sort_values(by="Score", ascending=False))

# Show selected features
selected_features = X.columns[selector.get_support()]

print("\nTop Selected Features:\n")
print(selected_features)
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q13.py
Feature Scores:

          Feature      Score
2      Attendance  27.020197
3  Previous_Score  12.246182
4     Final_Score  10.384788
1     Study_Hours   1.523740
0             Age   0.754839

Top Selected Features:

Index(['Attendance', 'Previous_Score', 'Final_Score'], dtype='str')

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
```

20) Decision Tree and Random Forest Classification (Iris Dataset)

Code

```
# Decision Tree Classifier (Iris)
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load data
data = load_iris()
X = data.data
y = data.target
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
# Create model
model = DecisionTreeClassifier(max_depth=4)
# Train model
model.fit(X_train, y_train)
# Check accuracy
accuracy = model.score(X_test, y_test)
print("Decision Tree Accuracy:", accuracy)
```

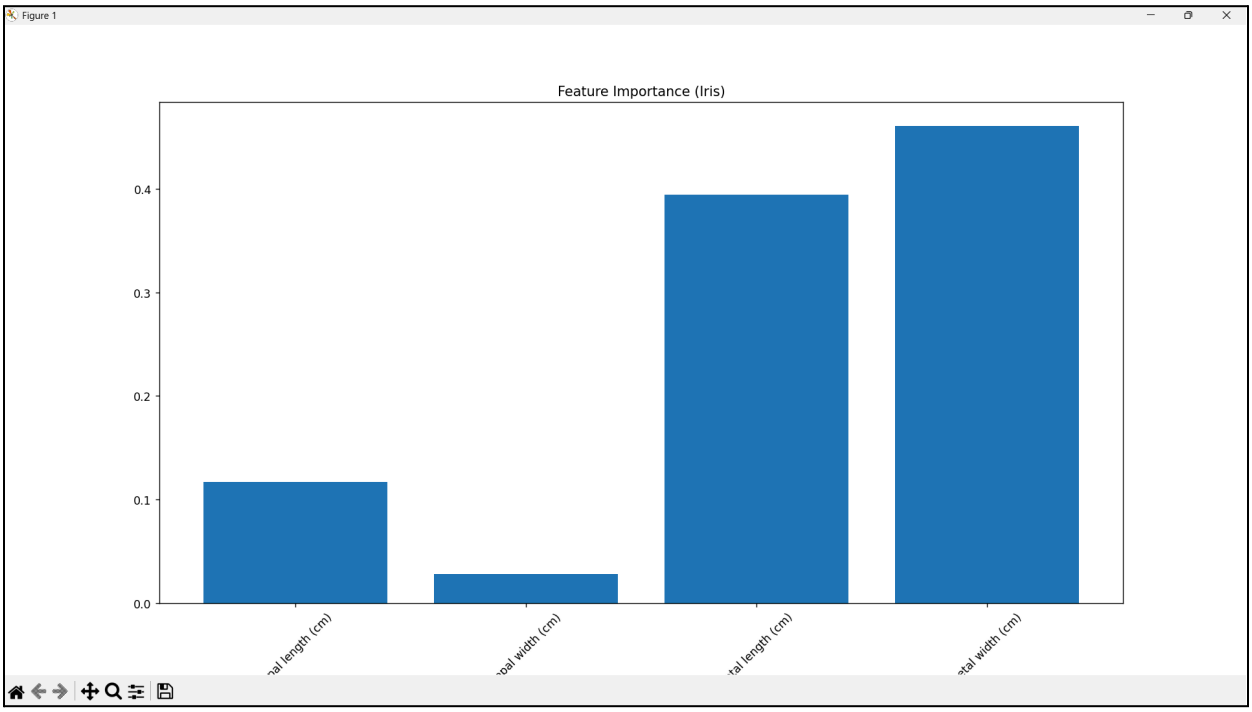
Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q14.py
Decision Tree Accuracy: 0.9333333333333333

haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$
```

Code

```
# Random Forest Classifier (Iris)
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load data
data = load_iris()
X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create model
model = RandomForestClassifier(n_estimators=100)

# Train model
model.fit(X_train, y_train)

# Accuracy
accuracy = model.score(X_test, y_test)
print("Random Forest Accuracy:", accuracy)

# Feature importance
importances = model.feature_importances_
features = data.feature_names

print("Feature Importance:", importances)

# Plot graph
plt.bar(features, importances)
plt.title("Feature Importance (Iris)")
plt.xticks(rotation=45)
plt.show()
```

Output




```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q15.py
Random Forest Accuracy: 0.9555555555555556
Feature Importance: [0.11706049 0.02798973 0.39422741 0.46072238]
```

21) Linear Regression Diabetes Detection.

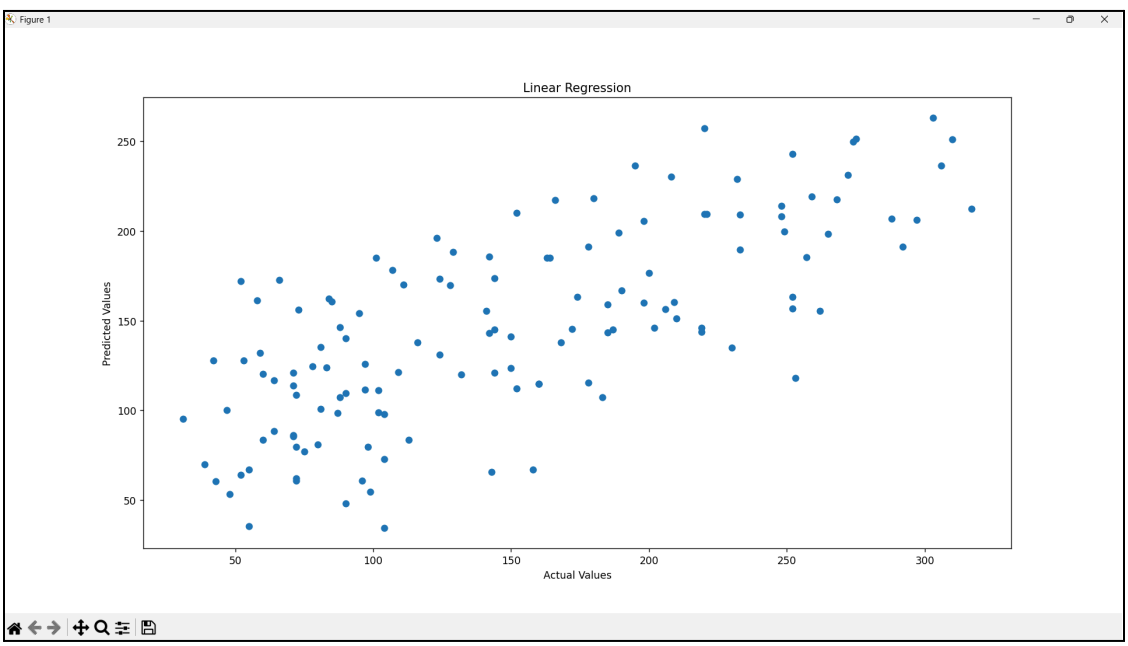
Code

```
# Linear Regression (All Features)
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
# Load dataset
data = load_diabetes()
X = data.data
y = data.target
# Split data
```

```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
# Create model
model = LinearRegression()
# Train model
model.fit(X_train, y_train)
# Predict
pred = model.predict(X_test)
# Accuracy
print("R2 Score:", r2_score(y_test, pred))
# Graph
plt.scatter(y_test, pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Linear Regression")
plt.show()
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q16.py
R2 Score: 0.5274753115483624
```



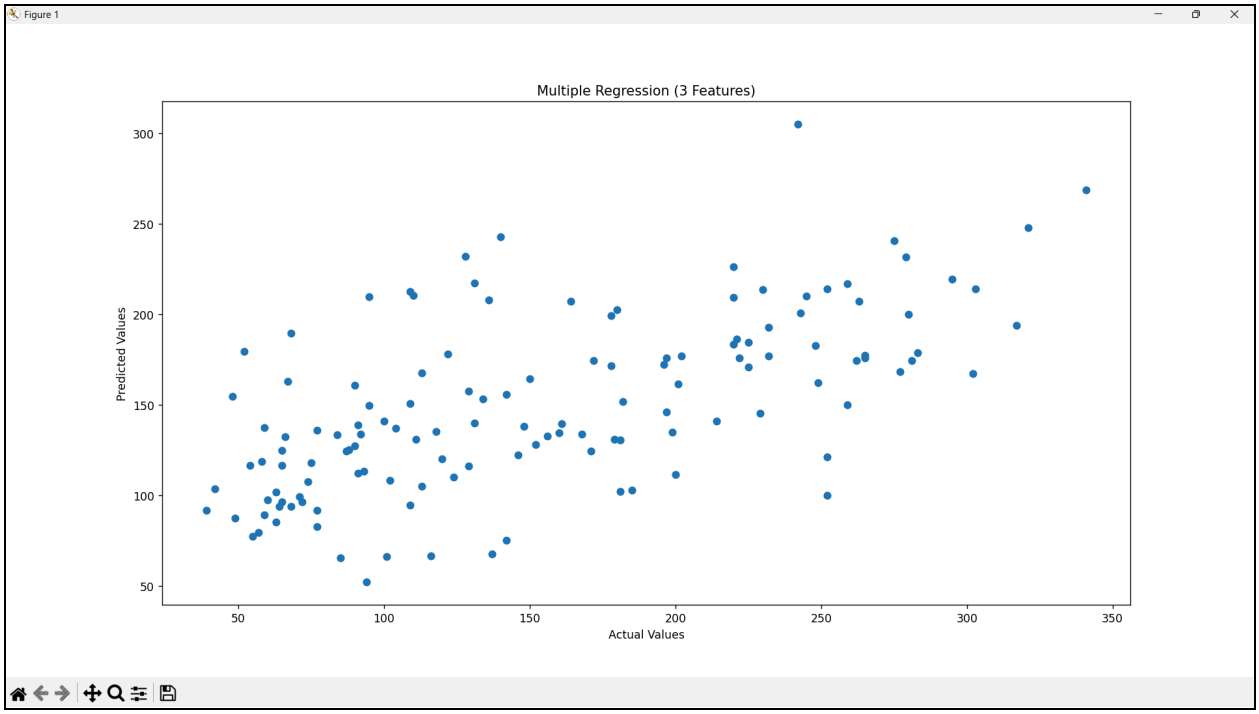
22) Multiple Linear Regression.

Code

```
# Multiple Regression (Only 3 Features)
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
# Load dataset
data = load_diabetes()
X = data.data
y = data.target
# Use only first 3 features
X = X[:, :3]
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
# Create model
model = LinearRegression()
# Train model
model.fit(X_train, y_train)
# Predict
pred = model.predict(X_test)
# Accuracy
print("R2 Score:", r2_score(y_test, pred))
# Graph
plt.scatter(y_test, pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Multiple Regression (3 Features)")
plt.show()
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q17.py
R2 Score: 0.38855091569385225
```



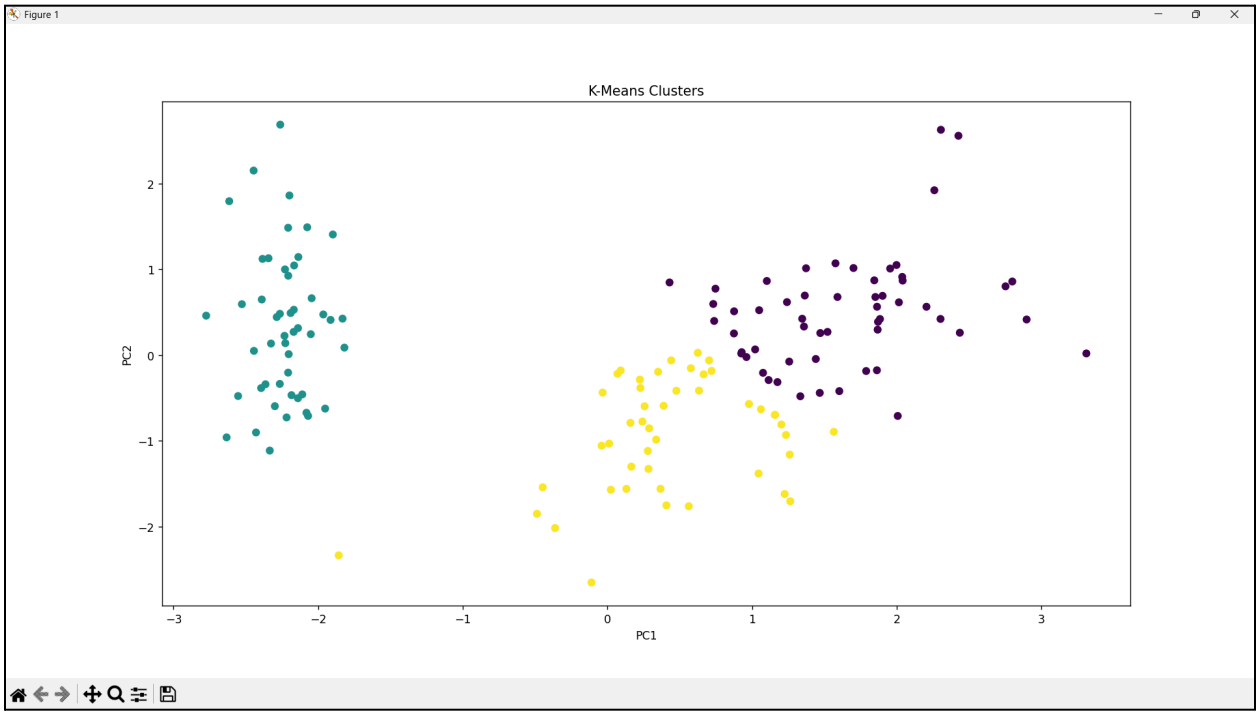
23) K - Means Clustering

Code

```
# K-Means Clustering (Iris)
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
# Load dataset
data = load_iris()
X = data.data
y = data.target   # only for comparison
# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Apply K-Means
kmeans = KMeans(n_clusters=3)
clusters = kmeans.fit_predict(X_scaled)
# Reduce to 2D (for graph)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Plot clusters
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters)
plt.title("K-Means Clusters")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Output

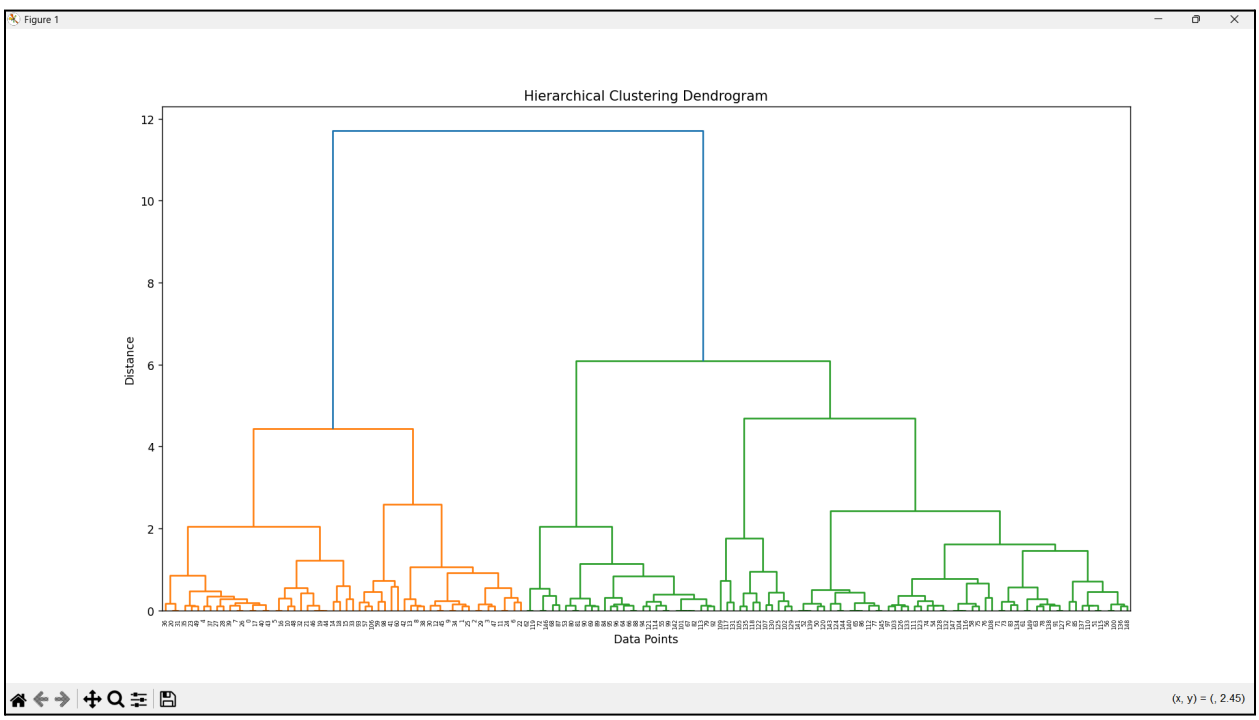


24) Hierarchical Clustering using Dendrogram (Iris Dataset)

Code

```
# Hierarchical Clustering (Dendrogram)
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from scipy.cluster.hierarchy import linkage, dendrogram
# Load dataset
data = load_iris()
X = data.data
# Apply clustering (using first 2 features for simplicity)
Z = linkage(X[:, :2], method='ward')
# Show first few values
print(Z[:5])
```

```python
# Plot dendrogram
plt.figure(figsize=(8,5))
dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

Output

```
haroo@LenovoIdeapad MINGW64 ~/Desktop/ML-PYTHON/experiments
$ python Q19.py
[[ 0. 17.  0.  2.]
 [ 2. 29.  0.  2.]
 [ 5. 16.  0.  2.]
 [11. 24.  0.  2.]
 [ 7. 26.  0.  2.]]
```